NEURAL WAVE AI HACKATHON 2026 · SFIDA LIFEWARE

The Neuron Migration

Analisi del problema, strumenti e strategia di soluzione

USI Lugano · 16–18 ottobre 2026
Documento di lavoro per il team

Punti chiave

1. È un problema di **sintesi di programmi da esempi input-output** con vincolo di esattezza: il machine learning guida la ricerca, non produce i valori.
2. Il primo strumento da costruire è il **diff harness**: confronto esatto con target e diagnostici, residui e contesto automatico per ogni riga.
3. Ogni convenzione scoperta (arrotondamento, calendario, epoca, data di riferimento dei parametri) diventa una riga di configurazione: **regole come dati**.
4. Le tecniche decisive: **ragionamento a intervalli** sui valori arrotondati, bisezione annuale in t2, verifica passo per passo in t3.
5. Esistono **etichette gratuite** oltre ai target: backtesting delle scadenze per t1, riscatti totali e switch per le quote di t3.
6. t5 si affronta solo a regole stabili, privilegiando la precisione: le righe isolate non spiegate sono probabilmente correzioni manuali.

Come è organizzato il documento. Quattro parti: il problema (dati, compiti, valutazione), l'analisi della complessità e degli indizi nascosti nel testo della sfida, gli strumenti e i paper utili, e la struttura proposta per la soluzione con un piano per le 48 ore. I numeri tra parentesi quadre rimandano all'elenco delle fonti in fondo al documento e sono cliccabili.

Indice

1. Il problema . . . 3
Come funziona una polizza unit-linked . . . 3
I dati . . . 4
I compiti . . . 5
Come vieni valutato . . . 5
2. Analisi del problema e della complessità . . . 6
Che tipo di problema è . . . 6
Le fonti di complessità . . . 6
Gli indizi nascosti nel testo . . . 7
Analisi compito per compito . . . 8
3. Strumenti e paper utili . . . 10
Software . . . 10
Paper . . . 11
4. Come strutturerei la soluzione . . . 13
I blocchi della soluzione . . . 13
Le tecniche che fanno la differenza . . . 14
Validazione: più etichette di quelle che ti danno . . . 15
Il codice consegnato . . . 15
Piano per le 48 ore . . . 16
Errori da evitare . . . 16
Fonti e link . . . 17

1. Il problema

Obiettivo in sintesi

Neuron, il sistema che dal 1994 calcola premi, costi, quote e pagamenti delle polizze unit-linked di NeuralWave Life, va sostituito da Lifeware. Il nuovo sistema può andare in produzione solo se riproduce i valori di Neuron per ogni polizza, al centesimo, ma non esistono né codice sorgente né documentazione completa. Neuron è nato in COBOL ed è stato in parte riscritto in Java (Neuron NG); i parametri sono cambiati molte volte e alcuni valori sono stati corretti a mano.

Hai l'export completo del sistema e, per il 60% delle polizze, i valori calcolati da Neuron. Devi ricostruire le regole e le formule che stanno in mezzo e scriverle come codice Python deterministico, che un revisore legale possa leggere e verificare. L'AI deve fare il detective, non la calcolatrice: un valore che si avvicina a quello giusto non vale nulla, perché ogni importo deve essere esatto, spiegabile e riproducibile.

Come funziona una polizza unit-linked

Il modo più semplice per capire tutti i compiti insieme è seguire la vita di un premio.

Il cliente paga, per esempio, 100 franchi. La compagnia trattiene un costo d'ingresso (entry charge), una percentuale che dipende anche dal tier del broker. Il broker che ha venduto la polizza matura una commissione. Il resto compra quote di uno o più fondi al prezzo di emissione, nel giorno di negoziazione (trade date), ripartito secondo l'allocazione scelta dal cliente. Se un fondo è in un'altra valuta serve un tasso di cambio. Le quote finiscono in uno di due contenitori: le quote iniziali (pot 1), comprate con i premi dei primi mesi, e le quote base (pot 2). I due pot hanno commissioni di gestione diverse.

A ogni anniversario mensile (lo stesso giorno del mese della data d'inizio) Neuron preleva tre costi cancellando quote in tutti i fondi e pot:

- la spesa amministrativa fissa;
- la commissione di gestione, una percentuale annua del valore presa mese per mese, diversa per pot;
- il premio di rischio, cioè il prezzo della copertura caso morte.

Il premio di rischio si calcola sul capitale sotto rischio, cioè la parte della prestazione caso morte che supera il valore delle quote. Questo capitale viene moltiplicato per la probabilità di morte entro un anno, presa da una tavola di mortalità per età e sesso, con un caricamento di sicurezza e un fattore maggiorativo per i fumatori.

Il valore della polizza è il numero di quote per il prezzo di riscatto (più basso di quello di emissione), convertito nella valuta della polizza. Se il cliente riscatta, riceve questo valore meno una penale, salvo nei periodi di esenzione (waiver). Può anche fare riscatti parziali o spostare denaro tra fondi (switch). Esistono poi accordi individuali che sostituiscono le tariffe standard per singole polizze. Infine ci sono gli stati della polizza: 10 attiva con premi, 20 liberata, 30 sospensione dei premi, 60 riscattata, 70 decesso.

Da ricordare: ogni passaggio ha la sua data

Firma dell'offerta, inizio e fine del contratto, scadenza della rata, addebito, arrivo dei soldi (money date), negoziazione delle quote, elaborazione nel sistema, valutazione. Gran parte delle regole nascoste consiste nel capire quale di queste date governa ciascun calcolo.

I dati

L'export contiene 23 tabelle, che conviene pensare in cinque gruppi.

Gruppo	Tabelle	Contenuto
Polizza e persone	NRPOLH, NRRISK, NRPERS, NRBROK	Versioni della polizza con validità; assicurati con nascita, sesso, fumo; broker e tier
Prodotto e parametri	NRPROD, NRCHRG, NRENTC, NRCOMR, NRAGRE, NRSURR, NRWAIV, NRILLU	Tariffe con periodi di validità: costi, entry charge, commissioni, accordi individuali, penali, esenzioni, tassi di proiezione
Denaro e quote	NRPREM, NRALLO, NRBOOK	Rate attese e incassate; allocazione ai fondi; circa 8 milioni di movimenti di quote
Mercato	NRFUND, NRFPRC, NRFXRT, NRMORT	Fondi; prezzi di emissione e riscatto; cambi; tavole di mortalità
Eventi e metadati	NREVNT, NRCODE, NRDICT, NREXCP	Eventi di business; liste di codici; dizionario dati; correzioni manuali note

Formati. Sono quelli tipici di un export legacy.

- **Date:** sono numeri in formato CYYMMDD, comune nei sistemi IBM midrange. La prima cifra è il secolo (0 per il 1900, 1 per il 2000). Quindi 1250131 è il 31.1.2025, mentre il 15.3.1994 diventa 0940315, che salvato come numero perde lo zero e diventa 940315. Il decoder deve gestire numeri a 6 e a 7 cifre e capire quali valori speciali (per esempio 0 o 9999999) indicano "nessuna data" o "validità aperta".
- **Numeri:** sono a virgola fissa con decimali impliciti, da leggere nel dizionario NRDICT.
- **Codici:** molti campi vanno decodificati con NRCODE, e tradiscono l'origine svizzero-tedesca del sistema. LS sta con ogni probabilità per Lastschrift, legato al Lastschriftverfahren (LSV), l'addebito diretto la cui infrastruttura è gestita da SIX [3], e UB per Überweisung (bonifico). E e L corrispondono probabilmente a Einmalprämie (premio unico) e laufende Prämie (premio periodico).

Periodo e divisione. Le polizze vanno dal 1994 al 2026 e attraversano gli anni del COBOL, la conversione e molti anni di Neuron NG. La divisione è:

- 60% di polizze di train, con dati e target;
- 20% pubbliche, con dati ma senza target, valutate sulla leaderboard live;
- 20% private, di cui non vedrai nemmeno i dati.

Diagnostici. Oltre ai target ci sono file che mostrano passaggi intermedi. Per t2, il valore proiettato a fine di ogni anno di polizza. Per t3, gli importi e le quote dei singoli prelievi mancanti, le quote per fondo e pot, e il valore prima della penale.

> ### Avvertimento sui movimenti (NRBOOK)
>
> NRBOOK serve per capire e fare debug, non come fonte delle formule. Gli ordini del cliente (switch, riscatti parziali e totali) sono completi e li puoi usare come input. I movimenti di premio e di costo invece li devi ricalcolare tu, perché dopo la migrazione non ci sarà più Neuron a scriverli.

I compiti

Task	Cosa devi calcolare	Chiave → valore
t0 Commissioni	Per ogni rata incassata e non stornata: tasso NRCOMR (per tier del broker e tipo E o L) × importo ricevuto PR_AMTRC. Il broker può cambiare negli anni	(polizza, n° rata) → importo
t1 Prossime scadenze	Polizze a premio periodico in stato 10 al 30.6.2026: tutte le scadenze dopo il 30.6.2026 e fino al 30.6.2027 incluso, ognuna con la sua data d'incasso. Nessuna scadenza dalla fine del contratto in poi; nulla cambia dopo l'estrazione	(polizza, n) → (scadenza, incasso)
t2 Valore proiettato a scadenza	Il valore finale dell'offerta firmata: simulazione mese per mese con crescita al tasso annuo di NRILLU, senza prezzi reali, valute o accordi individuali, con tutto il valore trattato come un unico importo	polizza → importo
t3 Valore di riscatto	Contabilità completa a quote e valore di riscatto alle date di valutazione (nel train 31.12.2006, 30.4.2020, 31.12.2025): quote × prezzo di riscatto, in valuta di polizza, meno penale. Zero se riscattata o se l'assicurato è morto	(polizza, data) → importo
t4 Estratto annuale	Per polizza e anno solare, dall'inizio al riscatto o decesso e al più tardi 2025: premi, entry charge, admin, gestione, rischio, totale	(polizza, anno) → 6 importi
t5 Outlier (bonus)	I valori di tutti i task sovrascritti a mano, anche quelli non in NREXCP. Ricevi i target delle polizze da controllare	insieme di (task, chiave), F1
t6 Strumenti (bonus)	Visualizzazioni e tool usati per trovare e spiegare le regole	qualitativo

Come vieni valutato

Accuratezza. Una riga conta solo se è esatta: importo al centesimo, data al giorno, e nell'estratto annuale tutte e sei le colonne insieme. I valori corretti a mano sono esclusi dall'accuratezza. Il punteggio di ogni task è la quota di righe esatte sulle polizze pubbliche e private, con un bonus se arrivi al 100%.

Manutenibilità. È misurata con il maintainability index di radon, solo sui file dei task. L'indice combina volume di Halstead, complessità ciclomatica, righe di codice e percentuale di commenti [12].

Altri punteggi. Ci sono i bonus t5 (F1 contro tutte le correzioni manuali note) e t6. C'è un punteggio di generalizzazione: quanto lavoro servirebbe per applicare il tuo approccio a un target nuovo. Infine c'è la leaderboard pubblica.

Consegna. È un pacchetto Python con submission.py e una funzione per task. Ogni funzione riceve la cartella dei CSV e restituisce date e Decimal. Il codice gira offline sui dati privati, con un limite di tempo, e non può leggere i target (tranne in t5) né contenerne valori.

> ### Un dettaglio strategico
> La soluzione di riferimento degli organizzatori fa il 100% su tutti i task con meno di 300 righe di Python. Le regole sono poche e generali.

2. Analisi del problema e della complessità

Che tipo di problema è

Formalmente è un problema di sintesi di programmi da esempi input-output, con un vincolo di esattezza. Esiste una funzione deterministica sconosciuta (Neuron) e tu osservi molte coppie "dati della polizza → valore". Devi trovare un programma che coincida con essa su ogni riga, comprese quelle che non vedrai mai. Ne seguono tre conseguenze.

Il machine learning classico può solo guidarti.

- Una regressione che sbaglia di 0.004 fa cadere la riga.
- La funzione vera è discontinua: è fatta di arrotondamenti, salti di calendario e cambi di regime.
- Un modello statistico non estrapola a regimi, date o polizze diverse.

Ogni riga esatta è un'evidenza fortissima. Indovinare un importo al centesimo per caso è quasi impossibile. Conviene quindi contare le corrispondenze esatte invece di misurare errori medi, come si fa quando si decifra un codice conoscendo coppie di testo in chiaro e cifrato.

Gli strumenti industriali qui non bastano. Quasi tutti quelli per estrarre regole da sistemi legacy partono dal codice sorgente. Per esempio COBRAIN, presentato a EASE 2025, usa LLM con prompting few-shot per estrarre regole di business da programmi COBOL [2]. Qui il codice non c'è. Sei quindi nel territorio della regressione simbolica e della sintesi di programmi, cioè della ricerca automatica di formule e regole che spiegano i dati.

Le fonti di complessità

Fonte	Perché è difficile	Esempio
Formati legacy	Un errore di decodifica è silenzioso e si propaga ovunque	Date a 6 o 7 cifre, decimali impliciti, valori sentinella
Tempo e versioni	Ogni parametro vale "dal… al…": per ogni calcolo bisogna capire a quale data guardare	Tariffa alla data della firma, della rata o dell'elaborazione?
Regimi diversi	La funzione è a tratti: COBOL e Java, correzioni silenziose, tariffe che cambiano. "Una parte" riscritta implica che non tutti i moduli cambiano insieme	Lo stesso calcolo arrotonda diversamente prima e dopo una data
Convenzioni numeriche	Modo, precisione e momento dell'arrotondamento decidono il centesimo	Troncamento COBOL, half-up o half-even in Java, artefatti dei double, quote a 4 o 6 decimali, arrotondare per riga o sul totale
Convenzioni di calendario	Un giorno sbagliato fa cadere la riga	31 gennaio più un mese, anni bisestili, festivi ticinesi, base giorni 30/360 o effettiva, età all'ultimo compleanno o al più vicino
Dipendenza dal percorso	t3 è uno stato ricorsivo mese per mese per oltre trent'anni	Un milionesimo di quota sbagliato nel 1998 può spostare un centesimo nel 2025
Identificabilità	I valori aggregati ammettono più spiegazioni	Vedi l'esempio qui sotto
Rumore o regola?	Una correzione manuale e una regola mancante producono lo stesso sintomo	NREXCP è incompleta: hai solo alcuni positivi etichettati

Fonte	Perché è difficile	Esempio
Vincoli di generalizzazione	Polizze private mai viste, date di valutazione passate come parametro, scadenze nel 2027, esecuzione offline a tempo	8 milioni di movimenti, calendario festivo calcolato, nessun LLM a runtime

Un esempio di identificabilità

È ricavato dalle righe d'esempio del PDF della sfida, ed è una mia ipotesi.

- Nell'estratto 2021 di BL10000013 i premi valgono 2835.00, cioè esattamente 12 × 236.25.
- La commissione della rata 1 della stessa polizza è 9.45, cioè il 4% di 236.25.
- Le spese admin sono 78.00 = 12 × 6.50.
- L'entry charge 103.95 è 11 × 9.45. Sembrerebbe un 4% su 11 premi, con un premio finito nell'anno dopo per effetto della data di negoziazione.

Ma 103.95 è anche 7 × 9.45 + 5 × 7.56, cioè sette premi al 4% e cinque al 3.2%. Due storie diverse, stesso numero. È per questo che i diagnostici granulari valgono oro: rendono ogni ipotesi verificabile da sola.

Gli indizi nascosti nel testo

Il PDF della sfida suggerisce, senza dirle, dove stanno le regole.

Indizio nel testo	Regola probabile da testare
"Parametri validi alla data della domanda e non oggi"	Tariffe bloccate alla data della domanda (PH_APPDT); da verificare tabella per tabella, non per tutte insieme
"Dati della polizza con una storia", "il broker può cambiare"	Lookup "as-of" (il valore valido a una certa data) su NRPOLH; per t2 probabilmente la versione valida alla firma
"COBOL e Java non calcolano tutto allo stesso modo"	Una o più date di switch, forse diverse per modulo
"Le scadenze passate in PR_DUEDT mostrano come si comporta lo scadenzario"	Ancora della serie: inizio + n periodi, oppure rata precedente + un periodo, con la tipica deriva 31 → 28 → 28; nuova ancora dopo cambi di frequenza o stato
"Giorni al prossimo compleanno"	Convenzione dell'età nel premio di rischio: ultimo compleanno o più vicino, calcolata a inizio anno o ogni mese
"I centesimi dell'importo"	Modo di arrotondamento: le righe con mezzo centesimo esatto separano half-up, half-even e troncamento
"Festivi", "conteggio dei giorni"	Calendario locale per incassi e negoziazioni; base giorni per crescita o costi
"Data di elaborazione"	Il regime dipende da quando Neuron ha elaborato il movimento, non da quando è avvenuto l'evento
"Ultimo prezzo il giorno stesso o prima"	Lookup as-of sui prezzi; da verificare se vale anche per i cambi
"Premi per data valuta, entry charge per data di negoziazione"	Movimenti a cavallo d'anno che finiscono in anni diversi
"Riferimento sotto le 300 righe"	Le regole sono poche e generali: se accumuli eccezioni, stai modellando rumore

Analisi compito per compito

t0 – Commissioni

La formula è banale; la difficoltà sta nei lookup. Devi:

- escludere le rate stornate (evento 900), abbinando lo storno alla rata giusta;
- trovare il broker valido alla data giusta e il suo tier;
- scegliere il tasso NRCOMR valido alla data giusta e il tipo E o L;
- scegliere l'arrotondamento giusto.

Un trucco rapido: calcola il rapporto commissione/importo su tutte le righe. I rapporti si raggruppano attorno a pochi valori, che sono le tariffe. Le righe fuori da ogni gruppo indicano un lookup sbagliato o una correzione manuale. Prova anche l'arrotondamento svizzero ai 5 centesimi: costa un minuto.

t1 – Prossime scadenze

Ci sono due sottoproblemi.

Il primo è lo scadenzario: generare le date alla frequenza giusta. Devi capire come Neuron gestisce i giorni dal 29 al 31 e gli anni bisestili, e da quale data parte per calcolare le scadenze successive.

Il secondo è la data d'incasso. Per l'addebito diretto è probabilmente la scadenza spostata al primo giorno lavorativo. Può essere spostata solo in avanti, oppure tornare indietro se si cambierebbe mese, o anticipata di qualche giorno lavorativo, come nei sistemi reali dove l'ordine LSV+ deve partire almeno tre giorni bancari prima dell'incasso [4].

Una heatmap a calendario delle date d'incasso passate ti mostra a colpo d'occhio quali giorni Neuron considera festivi.

Il calendario ticinese. Ha una sottigliezza: il Venerdì Santo è festivo quasi ovunque e per le banche, ma non in Ticino. Il 26.3.2027 è esattamente il tipo di giorno che fa sbagliare una riga. Nella finestra cadono in giorno feriale anche:

- l'8 e il 25 dicembre 2026;
- l'1.1.2027, che è un venerdì (una rata del 1° gennaio passerebbe a lunedì 4);
- il 6 gennaio, il 19 marzo e il lunedì di Pasqua 29 marzo;
- l'Ascensione 6 maggio, il lunedì di Pentecoste 17 maggio e il Corpus Domini 27 maggio;
- il 29 giugno.

Nota di dominio: con LSV+ il cliente può opporsi all'addebito, e lo storno è possibile fino a 68 giorni di calendario dopo l'accredito [3]. È il meccanismo reale dietro le rate "tornate dalla banca" dell'evento 900.

t2 – Valore proiettato

È una simulazione senza rumore di mercato, quindi il laboratorio ideale per i costi. I gradi di libertà principali:

- quale versione della polizza usare: probabilmente quella valida alla firma, ignorando cambi successivi di premio, frequenza o scadenza;
- quali parametri: quelli validi alla domanda;
- come convertire il tasso annuo in mensile: $(1+i)^{1/12} - 1$ oppure $i/12$;
- l'ordine delle operazioni nel mese;
- se arrotondare a ogni passo;
- quale commissione di gestione vale per "un solo importo";
- la convenzione d'età.

Il file con i valori annuali permette una bisezione: trova il primo anno in cui diverge e lì c'è la regola mancante.

t3 – Valore di riscatto

È il cuore del problema: un motore contabile che elabora gli eventi in ordine di data. Deve gestire:

- l'investimento dei premi: netto dell'entry charge, allocazione valida a quella data, prezzo di emissione, cambio, pot 1 o 2;
- i prelievi mensili per fondo e pot al prezzo di riscatto;
- gli ordini del cliente presi da NRBOOK;
- l'esclusione di storni e correzioni tecniche;
- la valutazione all'ultimo prezzo disponibile, la penale e le esenzioni.

Le difficoltà sono la dipendenza dal percorso e la precisione su decenni.

Un trucco per trovare la data della conversione: conta le cifre decimali significative delle quote negli ordini del cliente (che sono completi), ordinati per data di elaborazione. Il salto di precisione probabilmente ti dà lo switch COBOL → Java. Cercherei anche un'eventuale riconversione una tantum delle quote al momento della migrazione.

Le date dei prelievi le devi calcolare tu: le righe di NRBOOK servono solo a verificarle. E t3 riceve le date di valutazione come parametro: nella valutazione privata potrebbero essere diverse.

t4 – Estratto annuale

È un'aggregazione di t3, ma con basi temporali diverse per colonna: data valuta per i premi, data di negoziazione per l'entry charge, data del prelievo per gli altri costi. In cambio è un ottimo controllo incrociato, perché le somme annuali vincolano i prelievi di t3.

t5 – Outlier

Quando le regole spiegano quasi tutto, ciò che resta è rumore umano oppure regole mancanti.

NREXCP ti dà alcuni esempi positivi. Studiane la firma (task, entità della differenza, importi tondi, date) e cercala tra i residui non spiegati. È un problema positive-unlabeled: hai solo alcuni esempi positivi certi, e tutto il resto è non etichettato.

Un segnale forte di correzione manuale è un valore isolato che devia mentre il resto della polizza è coerente. Per esempio un valore finale di t2 sbagliato con traiettoria annuale giusta, o un valore di riscatto sbagliato con quote coerenti.

Con l'F1 conviene privilegiare la precisione e affrontare t5 solo a regole stabili. Verifica anche se le correzioni di NREXCP vanno applicate come input: fanno parte dell'export, quindi è lecito.

3. Strumenti e paper utili

Tutto ciò che è pesante (PySR, LLM, solver) resta nella fase di scoperta. La consegna deve girare offline e veloce. Sotto ogni strumento trovi il nome del pacchetto da installare.

Software

Area	Strumento	A cosa ti serve qui
Dati	**Polars, DuckDB** `polars, duckdb`	Caricano gli 8 milioni di movimenti in secondi e hanno join temporali as-of nativi, perfetti per versioni, tariffe, prezzi e cambi
Dati	**decimal** `libreria standard`	Tutti i modi di arrotondamento (HALF_UP, HALF_EVEN, DOWN…) e contesti a precisione fissa per emulare il COBOL; per emulare un Java a double, float usato di proposito
Dati	**holidays** `holidays`	Festivi svizzeri per singolo cantone, Ticino compreso, generati al volo [5]: funziona offline anche per il 2027
Dati	**Desbordante** `desbordante`	Scopre e valida dipendenze funzionali esatte, approssimate e condizionali, oltre alle dipendenze di inclusione, con binding Python [6]: quali colonne determinano un tasso, quali chiavi collegano le 23 tabelle
Dati	**VisiData, ydata-profiling** `visidata, ydata-profiling`	Esplorazione rapida dei CSV e profili automatici
Regole	**scikit-learn + dtreeviz** `scikit-learn, dtreeviz`	Un albero che predice “riga sbagliata”: le sue soglie sono le condizioni nascoste
Regole	**imodels** `imodels`	Modelli a regole compatibili con scikit-learn, tra cui RuleFit, Skope-rules e FIGS [7], traducibili in codice leggibile
Regole	**wittgenstein** `wittgenstein`	RIPPER: liste di regole ordinate e compatte
Regole	**ruptures** `ruptures`	Rilevamento dei punti di cambiamento sui residui nel tempo: trova le date di switch
Formule	**PySR** `pysr`	Regressione simbolica con perdite personalizzate e template che vincolano la forma delle espressioni, su un motore in Julia [8]; solo in fase di scoperta
Formule	**Z3** `z3-solver`	Risolutore SMT: esiste un tasso unico compatibile con tutti i valori arrotondati di un gruppo? In aritmetica esatta
Formule	**statsmodels, scikit-learn, fractions** `statsmodels, scikit-learn`	Recupero robusto dei tassi (Huber, RANSAC); riconoscere costanti razionali come 1/12
Ricerca con LLM	**OpenEvolve** `openevolve`	Versione open-source di AlphaEvolve: un ensemble di LLM propone modifiche, un pool di valutatori le misura, un database conserva la storia dei programmi [9]. Il tuo valutatore è la quota di righe esatte
Ricerca con LLM	**ShinkaEvolve** `da GitHub (SakanaAI)`	Alternativa di Sakana AI orientata all’efficienza, che ha raggiunto lo stato dell’arte con circa 150 programmi valutati [10]: utile se ogni valutazione costa minuti
Outlier	**PyOD, pulearn** `pyod, pulearn`	Isolation Forest e affini; apprendimento positive-unlabeled con NREXCP come positivi

Area	Strumento	A cosa ti serve qui
Dominio	**lifelib** `lifelib`	Modelli attuariali open-source in Python, pensati anche per validazione e modernizzazione; la libreria savings contiene modelli come CashValue_ME [11], un riferimento per la struttura di una proiezione
Qualità	**radon** `radon`	Il maintainability index in CI, accanto all'accuratezza [12]
Visualizzazione	**Plotly, calplot, Streamlit o marimo, PM4Py** `plotly, calplot, streamlit, marimo, pm4py`	Waterfall per t4, timeline di polizza, heatmap a calendario per t1, dashboard dei residui; PM4Py per esplorare il log degli eventi

Un agente per il ciclo di scoperta

Per far girare il ciclo di scoperta durante il weekend, un agente di coding come Claude Code è un buon alleato. Può lanciare l'harness, leggere i residui, proporre una variante di regola, testarla e aggiornare il registro delle ipotesi, mentre voi decidete cosa entra nella consegna. Rende al meglio se esiste un comando unico che stampa la quota di righe esatte per task.

Paper

Paper	Perché ti serve
Wang et al., "Hypothesis Search: Inductive Reasoning with Language Models", ICLR 2024	L'LLM formula più ipotesi astratte in linguaggio naturale, le implementa come programmi Python e le verifica sugli esempi osservati [13]: è esattamente il tuo ciclo
Shojaee et al., "LLM-SR", ICLR 2025	Presentato come oral: l'LLM propone scheletri di equazioni usando la conoscenza di dominio, i parametri si stimano sui dati, dentro una ricerca evolutiva [14]
Romera-Paredes et al., "FunSearch", Nature 2024; Novikov et al., "AlphaEvolve", 2025	Il paradigma "LLM generatore + valutatore automatico + evoluzione"
Lange et al., "ShinkaEvolve", ICLR 2026	Ricerca evolutiva efficiente con poche valutazioni [10]
Cranmer, "Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl", 2023	Teoria e pratica della regressione simbolica moderna
Udrescu e Tegmark, "AI Feynman", Science Advances 2020	Scomporre il problema prima di cercare la formula, come scomporre t3 passo per passo
Ernst et al., "The Daikon system for dynamic detection of likely invariants", 2007	Inferire invarianti da tracce di esecuzione: ciò che fai con NRBOOK e i diagnostici
Gulwani, "Automating string processing in spreadsheets using input-output examples", POPL 2011	Programming by example su un linguaggio ristretto: il modello mentale della griglia di convenzioni
Truong, Oudre, Vayatis, "Selective review of offline change point detection methods", 2020	La teoria dietro ruptures
Abedjan, Golab, Naumann, "Profiling relational data: a survey", VLDB Journal 2015	Dipendenze funzionali e di inclusione per decifrare lo schema

Paper	Perché ti serve
Liu, Ting, Zhou, “Isolation Forest”, 2008; Elkan e Noto, “Learning classifiers from only positive and unlabeled data”, 2008	Le basi per t5
Goldberg, “What every computer scientist should know about floating-point arithmetic”, 1991	Riconoscere e riprodurre gli artefatti di un Java a double
Dickson, Hardy, Waters, “Actuarial Mathematics for Life Contingent Risks”	Il capitolo sui contratti equity-linked spiega costi, quote e capitale sotto rischio
Coleman et al., IEEE Computer 1994	Il maintainability index nasce nel 1994 come formula di regressione polinomiale, valutata su sistemi industriali [15]; la documentazione di radon lo definisce ancora molto sperimentale [12], quindi trattalo come vincolo, non come obiettivo

4. Come strutturerei la soluzione

Il principio che guida tutto è separare due mondi.

- **La scoperta** è offline, e lì puoi usare qualunque cosa: ML, LLM, solver.
- **La consegna** sono poche centinaia di righe deterministiche e senza rete, in cui le convenzioni scoperte sono dati e non rami di codice.

Lo schema mostra come si parlano.

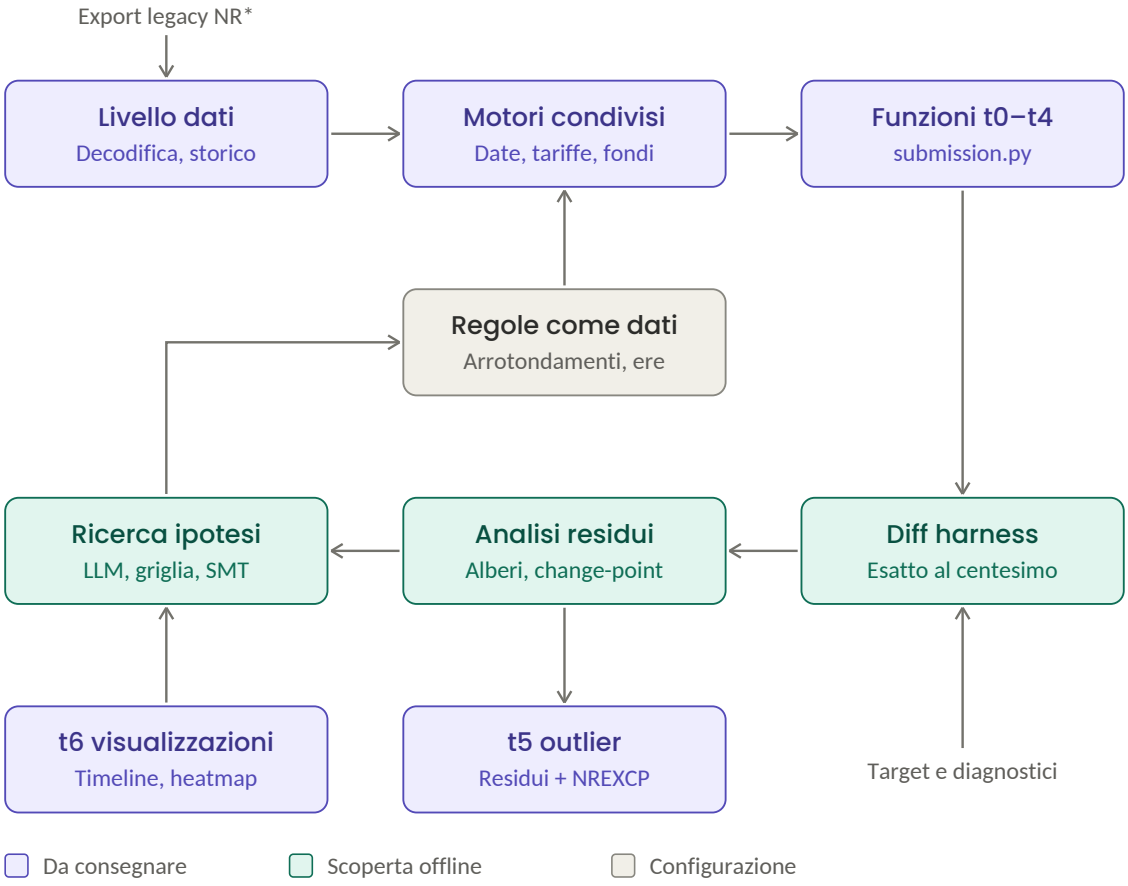


Figura 1 – Architettura: in viola ciò che si consegna, in verde il ciclo di scoperta offline, in grigio la configurazione.

I blocchi della soluzione

Livello dati. È un solo caricatore generico guidato da NRDICT: tipi, decimali impliciti, formato delle date e codici si leggono dal dizionario, non si scrivono a mano colonna per colonna. Sopra c'è un modello temporale con un'unica operazione, "il valore valido al giorno X", usata per versioni di polizza, tariffe, allocazioni, prezzi e cambi. Si aggiunge la pulizia degli eventi (coppie evento-storno, correzioni tecniche). Il risultato è una timeline per polizza, che è anche il primo strumento per t6.

Motori condivisi. Sono cinque:

- il calendario: mesi con regola di fine mese configurabile, giorni lavorativi ticinesi, anniversari mensili;
- il risolutore di parametri;
- la politica numerica;
- il motore dei costi, che t2 usa in forma semplificata e t3 completa;
- il motore contabile a quote, di cui t4 è un'aggregazione.

Regole come dati. Ogni scoperta diventa una riga di configurazione, non un if. Per esempio "importi troncati fino alla data X, half-up dopo" oppure "età al compleanno più vicino". Il revisore legge una tabella invece di un labirinto di condizioni. La complessità ciclomatica resta bassa. E ogni riga diventa una manopola che la ricerca automatica può girare.

Diff harness. È lo strumento più importante, e va scritto per primo. Per qualunque file target o diagnostico produce lo stesso output: esatta o no, residuo, e un contesto calcolato automaticamente. Il contesto comprende date chiave, prodotto, tier, epoca per data di elaborazione, frequenza, valuta, stato, età, giorni al compleanno, giorno della settimana, festivo e centesimi.

Analisi dei residui. Lavora su questa tabella, partendo dalle firme delle differenze:

- ±0.01 indica un arrotondamento;
- un rapporto costante tra calcolato e atteso indica un tasso sbagliato;
- una differenza pari a un parametro (una spesa admin, un premio) indica un mese di troppo o di meno;
- una differenza legata ai giorni al compleanno indica la convenzione d'età.

Alberi decisionali e liste di regole, addestrati a riconoscere le righe sbagliate, trovano le condizioni. I punti di cambiamento nei residui nel tempo trovano gli switch.

Ricerca di ipotesi. Chiude il ciclo con tre strumenti:

- una griglia sulle convenzioni, cioè tutte le combinazioni dei valori possibili di ogni manopola;
- il recupero dei parametri ignoti con intervalli, regressione robusta o Z3;
- un LLM che legge il report dei residui, propone ipotesi, le traduce in varianti di configurazione e le fa giudicare all'harness.

Le tecniche che fanno la differenza

Il ragionamento a intervalli

È la tecnica più potente per l'esattezza. Ogni valore arrotondato è un vincolo. Se una commissione attesa è 9.45 e vale round(r × A, 2) con half-up, allora r × A sta tra 9.445 e 9.455, quindi r sta in un intervallo preciso.

Intersecando gli intervalli di tutte le righe di un gruppo (stesso tier, tipo e periodo) ottieni il tasso esatto, oppure un'intersezione vuota. Un'intersezione vuota è informazione pura. Può voler dire che:

- il gruppo è definito male, perché manca una condizione;
- l'arrotondamento è sbagliato (con il troncamento l'intervallo diventa tra 9.45 e 9.46);
- una riga è un outlier.

È un test molto più severo di qualunque regressione. Per catene di calcolo più lunghe lo estendi con Z3.

La scomposizione. In t2 usi la bisezione sui valori annuali. In t3 conviene ripartire dallo stato vero dato dai diagnostici e verificare un passo alla volta, invece di simulare trent'anni e guardare solo la fine. Quando ogni passo è giusto dato lo stato giusto, l'intera simulazione torna.

La scomposizione è anche ciò che rende la ricerca trattabile. Otto manopole da tre valori fanno 6.561 combinazioni: troppe se ogni prova simula t3 per intero, pochissime se ogni manopola si testa su un sottoproblema isolato.

Il contesto per l'LLM. Quello che rende di più è dargli:

- il dizionario dati;
- alcune righe sbagliate con il loro contesto;
- le statistiche dei residui;
- il registro delle ipotesi già provate, con l'esito.

Chiedigli più ipotesi alternative, e lascia sempre decidere all'harness. Quel registro diventa anche il cuore del README e la prova del metodo per il punteggio di generalizzazione.

Validazione: più etichette di quelle che ti danno

I target coprono il 60% delle polizze, ma i dati nascondono etichette gratuite anche sulle pubbliche.

Per t1 puoi fare backtesting. Fingi che l'estrazione sia avvenuta il 30.6.2025, genera le scadenze dell'anno dopo e confrontale con le PR_DUEDT reali, sulle polizze senza cambi in quella finestra.

Per t3 hai diverse verifiche:

- i movimenti di costo in NRBOOK hanno le date anche senza importi, quindi verificano la logica degli anniversari su tutte le polizze;
- un riscatto totale, essendo un ordine completo, rivela le quote esatte possedute quel giorno;
- gli switch fanno lo stesso per singolo fondo;
- le somme annuali di t4 vincolano i prelievi di t3.

Per la generalizzazione valida con fold per gruppi di polizze e con holdout per coorte, per esempio per anno di domanda. Usa la leaderboard pubblica solo come controllo, mai per regolare le regole.

Il codice consegnato

Struttura. submission.py con funzioni sottili che compongono i motori, pochi moduli piccoli e un file di convenzioni. Tutto il codice di scoperta resta fuori dal pacchetto, perché contano solo i file dei task.

Maintainability index. Nella formula di radon il volume di Halstead e le righe entrano tramite logaritmo, la complessità ciclomatica in modo lineare e i commenti attraverso un termine sinusoidale [12]:

$$MI = \max\left[\,0,\ 100 \cdot (171 - 5.2 \ln V - 0.23\,G - 16.2 \ln L + 50 \sin \sqrt{2.4\,C}) / 171\,\right]$$

con V volume di Halstead, G complessità ciclomatica totale, L righe di codice, C percentuale di commenti

In pratica:

- conviene accorciare il codice;
- conviene evitare catene di if;
- conviene scrivere docstring che enuncino la regola in linguaggio di business.

Le docstring sono esattamente ciò che il revisore vuole leggere. Il comportamento preciso del termine sui commenti dipende dall'implementazione, quindi misuralo con radon invece di indovinarlo.

Esecuzione. Usa Decimal ovunque, con contesti espliciti. Niente rete, LLM o Julia a runtime. Misura il tempo d'esecuzione sull'intero dataset fin dal primo giorno.

Piano per le 48 ore

L'hackathon si svolge all'USI di Lugano da venerdì 16 ottobre alle 17:00 a domenica 18 alle 16:00 [1].

Quando	Obiettivo	Risultato
Venerdì sera	Caricatore da NRDICT, decodifica date, timeline di una polizza, harness, baseline t0, primo invio alla leaderboard	Pipeline verificata end-to-end
Sabato mattina	t0 al 100%; t1 con backtesting e calendario ticinese	Due task chiusi, bonus perfect match
Sabato pomeriggio	t2 con bisezione annuale; nasce il motore dei costi	Costi validati senza rumore di mercato
Sabato sera	t3 passo per passo su movimenti e quote; t4 come aggregazione	Contabilità a quote funzionante
Domenica mattina	Chiusura t3/t4, t5 sui residui, refactor in regole come dati, README, visualizzazioni	Consegna stabile e documentata
Domenica fino alle 16	Freeze con almeno due ore di margine	Nessuna sorpresa all'ultimo invio

Se siete in quattro, dividerei così:

- una persona su dati, harness e infrastruttura;
- una su t0, t1 e calendario;
- una su t2 e motore dei costi;
- una su t3 e t4.

t5 e t6 nascono dal lavoro di tutti e si raccolgono la domenica. In tre, unirei i primi due ruoli. Tutti devono usare lo stesso harness e la stessa configurazione, così ogni scoperta migliora subito i numeri degli altri.

Errori da evitare

I più costosi

- usare NRBOOK come fonte di premi e costi nella consegna;
- lasciare float nel codice senza volerlo;
- leggere i target o incorporarne valori;
- piegare le regole per far tornare righe isolate, che probabilmente sono correzioni manuali: non contano per l'accuratezza, ma valgono punti in t5;
- regolare le regole sulla leaderboard pubblica;
- dimenticare le date a 6 cifre prima del 2000;
- scoprire il limite di tempo all'ultimo invio.

Con un harness solido e la disciplina di trattare ogni convenzione come un'ipotesi da misurare, è un problema che premia il metodo più della forza bruta. Buon hackathon!

Fonti e link

Le fonti web consultate per questo documento. I link sono cliccabili.

N.	Fonte	Link
[1]	USI, Neural Wave 2026	usi.ch – Neural Wave: AI hackathon returns to USI usi.ch – Neural Wave: programma e orari neuralwave.ch
[2]	COBRAIN, EASE 2025	conf.researchr.org – LLM Vs Rule-Based: the COBRAIN tool
[3]	ZKB, Lastschriftverfahren (LSV)	zkb.ch – Lastschriftverfahren
[4]	Microsoft Learn, LSV collection	learn.microsoft.com – Process an LSV collection (Swiss version)
[5]	python-holidays	python-holidays.readthedocs.io
[6]	Desbordante	pypi.org/project/desbordante github.com/Desbordante/desbordante-core
[7]	imodels	github.com/csinva/imodels
[8]	PySR	github.com/astroautomata/PySR Documentazione API di PySR
[9]	OpenEvolve	github.com/algorithmicsuperintelligence/openevolve pypi.org/p/openevolve
[10]	ShinkaEvolve	github.com/SakanaAI/ShinkaEvolve sakana.ai/shinka-evolve arXiv 2509.19349
[11]	lifelib	github.com/lifelib-dev/lifelib lifelib.io
[12]	radon, Introduction to Code Metrics	radon.readthedocs.io – intro
[13]	Hypothesis Search (ICLR 2024)	arXiv 2309.05660
[14]	LLM-SR (ICLR 2025)	arXiv 2404.18400 github.com/deep-symbolic-mathematics/LLM-SR
[15]	Studio con sezione sul maintainability index	arXiv 2607.02744